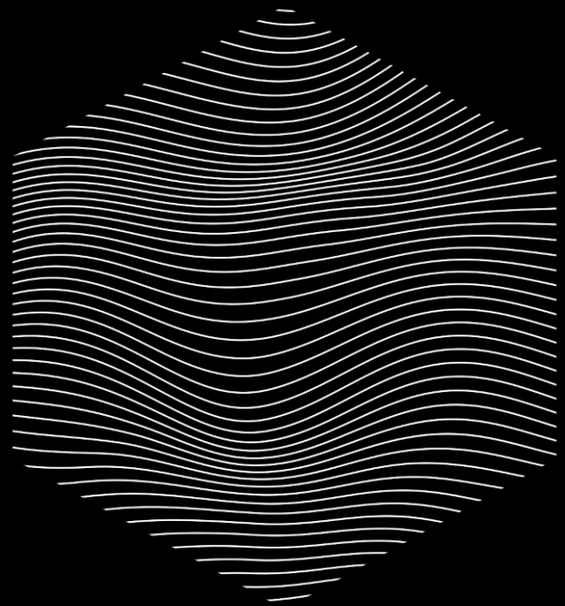
wingfoil

wingfoil



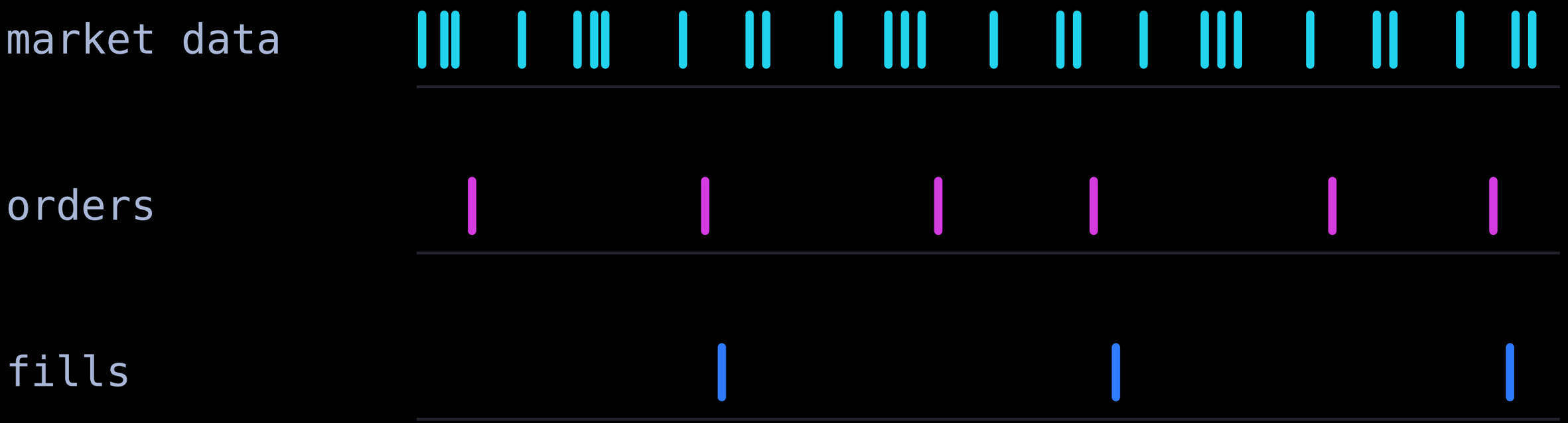
the ultra low latency data streaming framework

Jake Mitchell

August 2026

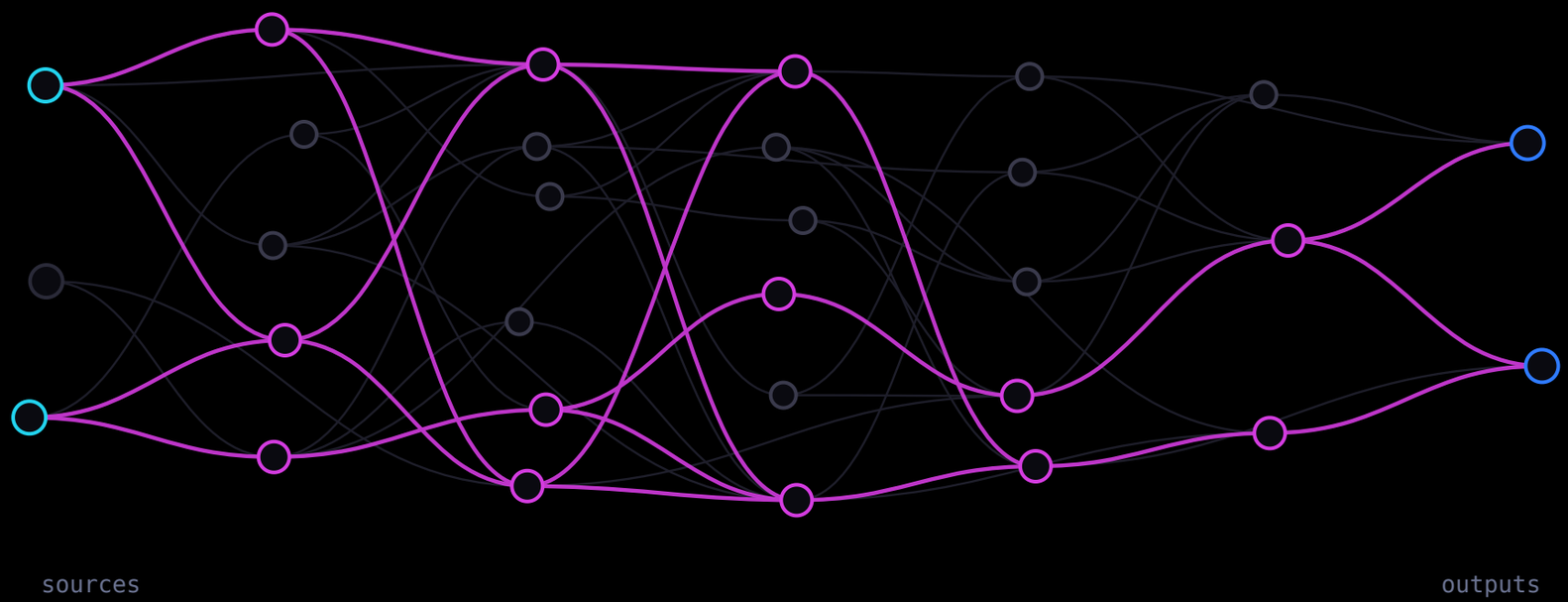
THE PROBLEM

Everything ticks at a different rate



THE SOLUTION

Wire a graph of calculations



- The framework abstracts over **what needs to tick, and when**.
- **Split and recombine** freely — it stays one graph.
- **Modulate the frequency** — window, throttle, sample.
- And it has to be **efficient**.

THE SURFACE

Wire it, build it, run it

```rust
let g = GraphBuilder::new();

let printed = g
    .ticker(Duration::from_millis(100))
    .count()
    .map(|i| format!("tick {i}"))
    .for_each(|msg: &String| { println!("  {msg}"); Ok(()) });

g.build().run(RunMode::HistoricalFrom(NanoTime::ZERO), RunFor::Cycles(5))?;
```

```
historical run (instant):
  tick 1
  tick 2
  tick 3
  tick 4
  tick 5
```

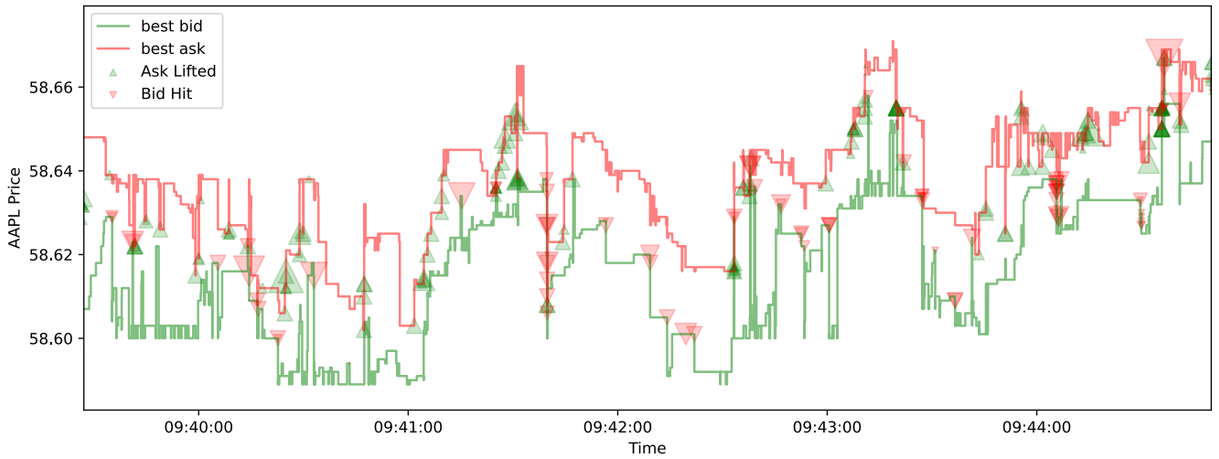
```
cargo run -p wingfoil --example hello_graph
```

MOTIVATION, NOT THESIS

A real graph: NASDAQ limit orders → book → prices + fills

```
let (fills, prices) =
    csv_read(&g, &src, get_time, true, None)?
        .map(move |chunk: &Burst<Message>| {
            process_orders(chunk, &book)
        })
        .split();

prices.filter_none().distinct()
      .csv_write(&prices_path)?;
fills.csv_write(&fills_path)?;
```



RUN MODES

Backtest it, then run it live

`RunMode::HistoricalFrom(NanoTime::ZERO)` — the whole hour

```
$ cargo run --release --example top_of_book
0.021_311  bid  585.33  ask  585.91  spread  0.58  mid  585.62
0.197_502  bid  585.33  ask  585.92  spread  0.59  mid  585.62
0.197_540  bid  585.33  ask  585.93  spread  0.60  mid  585.63
0.201_332  bid  585.36  ask  585.93  spread  0.57  mid  585.64
0.267_498  bid  585.73  ask  585.74  spread  0.01  mid  585.74
```

`RunMode::RealTime` — a three-second window of the same file · animation slowed 12×

```
$ cargo run --release --example top_of_book -- realtime
1,787,777,083.871_535  bid  585.33  ask  585.91  spread  0.58  mid  585.62
1,787,777,084.048_929  bid  585.33  ask  585.92  spread  0.59  mid  585.62
1,787,777,084.049_141  bid  585.33  ask  585.93  spread  0.60  mid  585.63
1,787,777,084.053_476  bid  585.36  ask  585.93  spread  0.57  mid  585.64
1,787,777,084.119_906  bid  585.73  ask  585.74  spread  0.01  mid  585.74
```

I/O

Sixteen adapters, three patterns

MESSAGING
Kafka · ZeroMQ · Fluvio · Redis · etcd

LOW-LATENCY TRANSPORT
Aeron · iceoryx2

TRADING
FIX

STORAGE & FILES
kdb+ · PostgreSQL · CSV · lines

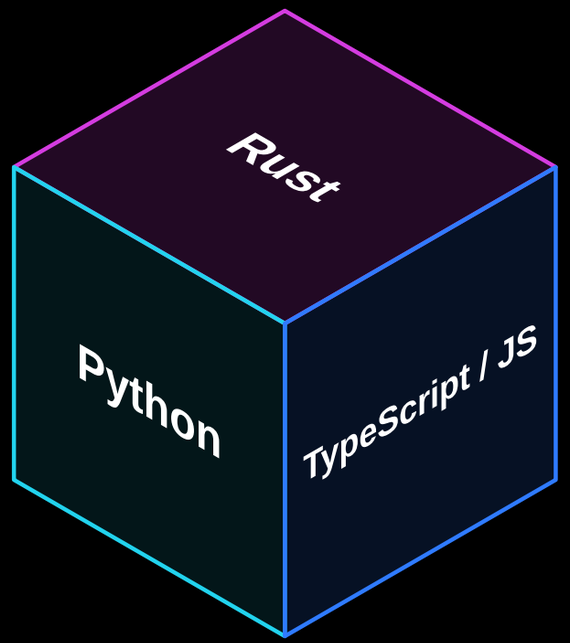
WEB
WebSocket client · WebSocket server

TELEMETRY
Prometheus · OpenTelemetry

Pattern	Latency	Example
Busy loop	nanoseconds	`iceoryx2`
Worker thread	microseconds	`ZeroMQ`
Async	microseconds	`kdb+`

SURFACES

Three languages, one engine



PART TWO

Nitro

Performance, the wall we hit chasing it,
and the pattern that got us through.

THE ENGINEERING PROBLEM

What we wanted Nitro to factor out

Per node, per cycle, interpreted	What we wanted instead
a `Box<dyn FnMut>` call	a direct call the optimiser can inline
a trip through a value slot	a local variable
a dirty-list walk to decide what runs	straight-line code, in topological order

THE FRONT DOOR

You write the wiring once, in plain Rust

```
wingfoil::nitro! {
    fn odds_evens(g: &GraphBuilder) -> Stream<String> {
        let count    = g.ticker(PERIOD).count();
        let is_even  = count.map(|i| i.is_multiple_of(2));
        let is_odd   = is_even.map(|b| !b);
        let odd_str  = count.filter(&is_odd).map(|i| format!("{i} is odd"));
        let even_str = count.filter(&is_even).map(|i| format!("{i} is even"));
        let labelled = odd_str.merge(&even_str);
        labelled
    }
}
```

…AND THIS IS WHAT IT BECOMES

One function. One local per node. No dyn.

```
let mut __k = Kernel::new(run_mode, run_for);

let mut __cfg_wf_anon_1   = PERIOD;
let mut __state_wf_anon_1 = __wf_op_ticker_seed_state(&__cfg_wf_anon_1);
let mut __v_wf_anon_1     = __wf_op_ticker_seed_value(&__cfg_wf_anon_1);
// … one cfg / state / value local per node, 11 nodes …

let mut __dirty = [false; 11usize];
while __k.begin_cycle(&mut __dirty) {
    let __t_wf_anon_1 = /* activation */ && {
        let mut __ctx = Ctx::new(&mut __k, 0usize);
        match __wf_op_ticker_cycle(&mut __cfg_wf_anon_1,
                                   &mut __state_wf_anon_1, (), &mut __ctx)
              .context("node 0 (wf_anon_1: ticker) cycle")? {
            Tick::Value(__val)  => { __v_wf_anon_1 = __val; true  }
            Tick::Silent(__val) => { __v_wf_anon_1 = __val; false }
            Tick::Quiet         => false,
        }
    };
    // … 10 more, emitted in topological order …
}
```

abridged · elisions marked · all 1,730 lines committed verbatim beside the example:
github.com/wingfoil-io/wingfoil → examples/core/dual_mode/expanded/

THE ONE DECISION EVERYTHING FOLLOWS FROM

Semantics became an associated function

```rust
trait Op {
    type Cfg;               // construction-time config; closures live here
    type State;             // engine-owned mutable state
    type In<'a>;            // typed inputs, passed in per cycle
    type Out;               // the produced value
    const ACTIVATION: Activation;

    fn cycle(cfg: &mut Self::Cfg, state: &mut Self::State,
             input: Self::In<'_>, ctx: &mut Ctx<'_>) -> Result<Tick<Self::Out>>;
}
```

> **Read what is *absent*. Nothing here says how state is stored, where inputs come from, or who calls it.**

THE SAME NODE, BOTH WAYS

The node changed completely. The wiring barely moved.

legacy — the node owns everything

```rust
#[node(active = [upstream], output = value: f64)]
impl MutableNode for ScaleStream {
    fn cycle(&mut self, _state: &mut GraphState)
        -> Result<bool>
    {
        self.value =
            self.upstream.peek_value() * self.factor;
        Ok(true)
    }
}
```

owns: its upstream handles, its state, its output slot

```rust
let count = ticker(Duration::from_millis(10))
    .count();
count.run(RunMode::RealTime, RunFor::Cycles(100))?;
```

now — the engine owns everything

```rust
#[op(build = scale, fluent)]
impl Op for Scale {
    type Cfg    = f64;              // config
    type State  = ();               // engine-owned
    type In<'a> = (&'a f64,);       // passed in
    type Out    = f64;
    const ACTIVATION: Activation = Activation::NONE;

    fn cycle(cfg: &mut f64, _state: &mut (),
             input: (&f64,), _ctx: &mut Ctx<'_>)
        -> Result<Tick<f64>>
    {
        Ok(Tick::Value(input.0 * *cfg))
    }
}
```

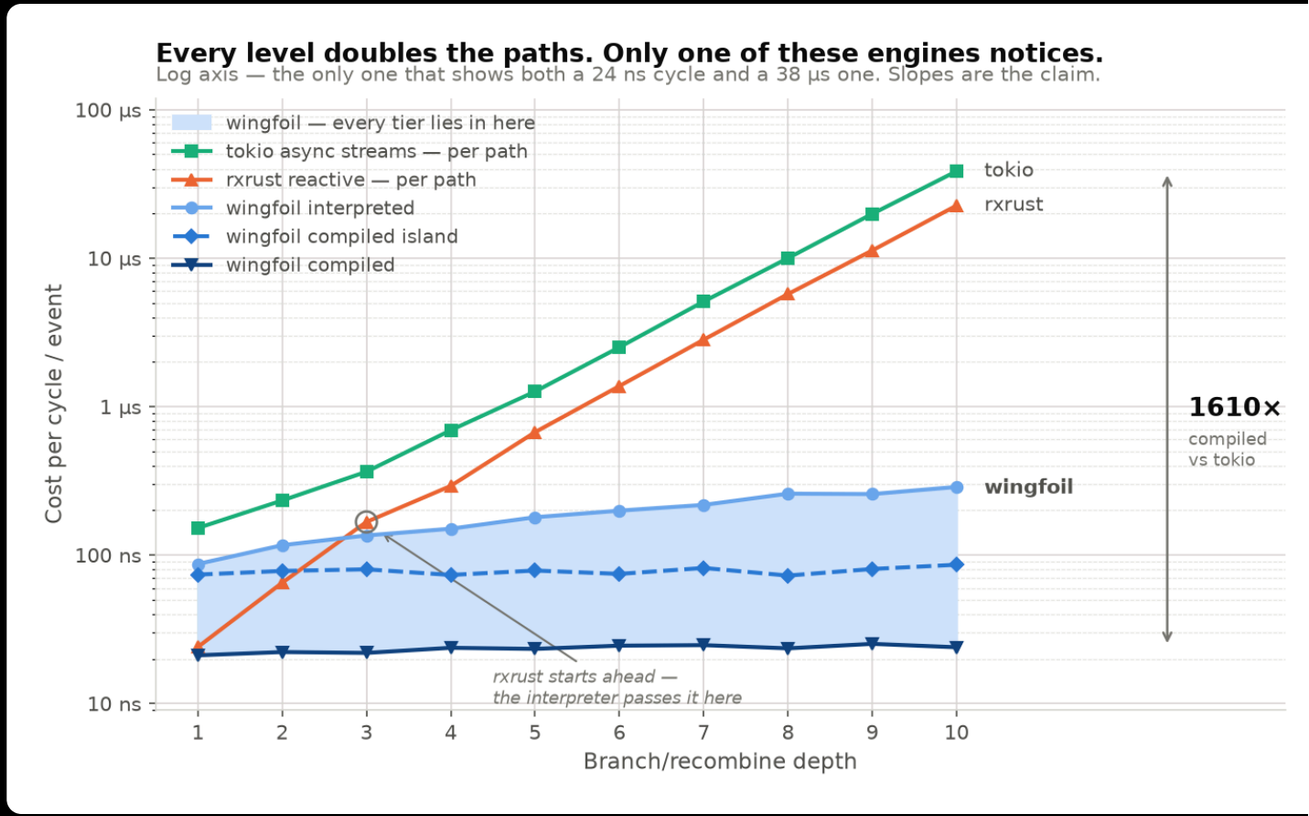
owns: nothing — cfg, state and inputs all arrive as arguments

```rust
let g = GraphBuilder::new();
let count = g.ticker(Duration::from_millis(10))
    .count();
g.build().run(RunMode::RealTime, RunFor::Cycles(100))?;
```

RESULTS

The 10×10 graph: 16.6 ns → 0.31 ns per node cycle

10×10 fan-out	per node cycle	
legacy engine	16.55 ns	
interpreted	11.64 ns	1.4×
compiled	**0.31 ns**	**53×**
nested island	1.39 ns	12×



PART THREE

What's next

Closer to the metal

1. **Verilog / FPGA** — the graph *as* gateware, with the backtest as its testbench.
2. **Core pin** — the graph thread on an isolated core.
3. **Kernel bypass** — Onload, then ef_vi/DPDK. This is the one that buys the wire-to-trade number we currently decline to claim.
4. **Zero-allocation I/O** — no allocator on the hot path.

A platform, not just an engine

- Simulated exchange
- OMS / EMS
- Position keeping
- Risk
- Trade booking
- Venue adapters

GET INVOLVED

Contributors and users

Contributors

- Around **20 people** so far.
- **Good first issues** that really are — each names the file to change and the code to copy.
- The bigger projects are separable, and each links to a design rather than a blank page.

Users

- An institution, a trading firm, or your own account.
- Where the docs lost you. The benchmark you don't believe.
- A project wingfoil would suit — or plainly wouldn't.

> **We want to hear from you.**

THANK YOU

wingfoil

github.com/wingfoil-io/wingfoil · Apache-2.0

```
cargo add wingfoil
```

hello@wingfoil.io